

```
1
/**
2
*****
3
* @file      startup_stm32f407vgtx.s
4
* @author    Auto-generated by STM32CubeIDE
5
* @Abstract : Startup script for STM32F407VGTX Device
6
* @version   V1.0.0
7
* @date      2020-02-14
8
*****
9
*
10
* OVERVIEW FOR BEGINNERS:
11
* =====
12
* This file is the very first code that runs when your STM32 microcontroller
13
* powers on or resets. Before your C code (main()) can run, several things
14
* must be set up at the hardware level. This file does those things:
15
*
16
* 1. Sets up the stack pointer (where function call data lives in RAM)
17
* 2. Copies initialized global variables from Flash (permanent storage)
18
*    into RAM (working memory), because RAM loses its contents on power loss
19
* 3. Zeroes out uninitialized global variables (the BSS segment)
20
* 4. Calls SystemInit() to configure clocks
21
* 5. Calls __libc_init_array() to run C++ static constructors
22
* 6. Jumps to your main() function
23
```

```

23 *
24
25 * It also defines the "vector table" – a lookup table of function pointers
26
27 * that tells the hardware WHERE to jump when an interrupt or exception fires.
28
29 *
30
31 * This file is written in ARM Thumb assembly. You don't need to edit it
32
33 * unless you are doing very low-level customization.
34
35 */
36
37
38 /* =====
39
40 * ASSEMBLER DIRECTIVES
41
42 * These are instructions TO the assembler (the program that converts this
43
44 * text into machine code), not instructions that run on the chip itself.
45
46 * ===== */
47
48
49 .syntax unified      /* Use "unified" ARM/Thumb syntax – a modern syntax that
50
51                      lets you write both ARM and Thumb instructions the
52
53                      same way, instead of having two different styles. */
54
55
56
57
58 .cpu cortex-m4      /* Tell the assembler we're targeting the ARM Cortex-M4
59
60                      core inside the STM32F407. This enables Cortex-M4
61
62                      specific instructions. */
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668

```



```

.global Default_Handler /* A catch-all handler for any interrupt that doesn't
69
                                have a real implementation written yet. */
70
71
/* =====
72
* LINKER SCRIPT SYMBOL REFERENCES
73
* The .word directive places a 32-bit address literal into the output binary.
74
* These symbols (starting with underscore) are NOT defined here – they are
75
* defined in the linker script (.ld file) which knows the exact memory map
76
* of this chip. The linker will fill in the actual addresses at link time.
77
*
78
* Think of this like declaring "extern" variables in C: we're telling the
79
* assembler "these addresses exist somewhere, the linker will resolve them."
80
* ===== */
81
82
.word _sidata /* Source address in Flash where the initial values of
83
                initialized global variables are stored.
84
                ("si" = source of initialized data) */
85
86
.word _sdata /* Start address in RAM where .data section begins.
87
                ("s" = start, "data" = initialized globals/statics) */
88
89
.word _edata /* End address in RAM where .data section ends.
90
                ("e" = end) */

```

```
91
92
.word _sbss    /* Start address in RAM of the .bss section.
93
                BSS = Block Started by Symbol – this is where uninitialized
94
                global and static variables live. C standard says they must
95
                start as zero, so we'll zero this region out explicitly. */
96
97
.word _ebss    /* End address in RAM of the .bss section. */
98
99
100
/* =====
101
    * RESET HANDLER
102
    * =====
103
    *
104
    * This is the entry point of your entire program. When the chip powers on
105
    * or is reset, the hardware automatically:
106
    *   1. Reads address 0x00000000 → loads it into the Stack Pointer (SP)
107
    *   2. Reads address 0x00000004 → loads it into the Program Counter (PC)
108
    *
109
    * Address 0x00000004 in the vector table contains the address of
110
    * Reset_Handler, so execution jumps here immediately after reset.
111
    *
112
    * .section .text.Reset_Handler
113
```

```

* Put this code in a special named section so the linker can place it
114
* correctly. The linker script will pull this into the .text region of Flash.
115
*
116
* .weak Reset_Handler
117
* "Weak" means: if another file defines a symbol with this same name,
118
* use that one instead and discard this one. This lets application code
119
* override the default startup behavior if needed.
120
*
121
* .type Reset_Handler, %function
122
* Tells the assembler (and debugger) that this symbol is a function,
123
* not just a data label. Important for correct stack unwinding in debuggers.
124
*/
125
126
.section .text.Reset_Handler
127
.weak Reset_Handler
128
.type Reset_Handler, %function
129
130
Reset_Handler:
131
132
/* --- Step 1: Initialize the Stack Pointer ---
133
*
134
* The stack is a region of RAM used to store:
135
* - Return addresses when calling functions

```

```
136
    *   - Local variables
137
    *   - Saved register values during interrupts
138
    *
139
    * On Cortex-M, the stack grows DOWNWARD in memory (from high address to low).
140
    * _estack is defined in the linker script as the TOP of RAM (highest address).
141
    * We load this address into SP (Stack Pointer register, also called r13).
142
    *
143
    * "ldr r0, =_estack" is a pseudo-instruction. The assembler generates a
144
    * PC-relative load from a literal pool – it's loading the 32-bit address
145
    * value of _estack into register r0.
146
    *
147
    * "mov sp, r0" copies that value into the actual Stack Pointer register.
148
    *
149
    * After this, any function call or interrupt will use this stack correctly.
150
    */
151
    ldr    r0, =_estack
152
    mov    sp, r0
153
154
155
    /* --- Step 2: Copy .data section from Flash to RAM ---
156
    *
157
    * WHY IS THIS NECESSARY?
158
```

```
159 * Flash memory retains its contents when power is removed, but it's
160 * read-only at runtime (you can't write to Flash during normal execution
161 * without a special erase/write procedure). RAM is fast and writable,
162 * but loses everything on power-off.
163 *
164 * When you write C code like:
165 *     int myGlobal = 42;    // initialized global variable
166 *
167 * The value 42 must be stored somewhere permanently (Flash), but the
168 * variable itself must live in RAM so it can be modified. The linker puts
169 * the initial value (42) into a Flash region called .sidata, and reserves
170 * a slot in RAM called .data. This startup code copies from .sidata → .data
171 * before main() runs, so your globals have their correct initial values.
172 *
173 * The copy loop below:
174 *
175 *     r0 = destination start (RAM: _sdata)
176 *     r1 = destination end   (RAM: _edata)
177 *     r2 = source start      (Flash: _sidata)
178 *     r3 = byte offset counter (starts at 0, increments by 4 each iteration)
179 *
180 * We jump to LoopCopyDataInit first to check whether there's anything
181 * to copy at all (handles the edge case of an empty .data section).
182 */
```



```

181     ldr r0, =_sdata    /* Load address of start of .data section in RAM */
182
183     ldr r1, =_edata    /* Load address of end of .data section in RAM */
184
185     ldr r2, =_sidata    /* Load address of initial values stored in Flash */
186
187     movs r3, #0        /* r3 = 0 (byte offset, starts at beginning) */
188
189     b LoopCopyDataInit /* Jump to the loop condition check first */
190
191
192     /* Loop body: copy one 32-bit word (4 bytes) per iteration */
193
194     ldr r4, [r2, r3]    /* Load 4 bytes from Flash: address = r2 + r3 offset */
195
196     str r4, [r0, r3]    /* Store those 4 bytes to RAM: address = r0 + r3 offset */
197
198     adds r3, r3, #4     /* Advance offset by 4 bytes (one word) */
199
200     /* "adds" sets condition flags (N, Z, C, V) – needed by bcc below */
201
202
203     LoopCopyDataInit:
204
205     /* Loop condition: check if we've reached the end of .data */
206
207     adds r4, r0, r3     /* r4 = current destination address (RAM base + offset) */
208
209     cmp r4, r1          /* Compare current address against end-of-.data (_edata) */
210
211     bcc CopyDataInit    /* BCC = Branch if Carry Clear = Branch if r4 < r1
212
213                         i.e., loop back if we haven't reached the end yet.
214
215                         When r4 >= r1, we fall through and the copy is done. */

```

```

204  /* --- Step 3: Zero-fill the .bss section in RAM ---
205  *
206  * WHY IS THIS NECESSARY?
207  * The C standard guarantees that uninitialized global and static variables
208  * start at zero. For example:
209  *
210  *     int counter;           // uninitialized global – must start as 0
211  *
212  *     static int flag;      // uninitialized static – must start as 0
213  *
214  * The linker places these in a section called .bss (historically "Block
215  * Started by Symbol"). Nothing is stored in Flash for .bss (there are no
216  * initial values to store – they're all zero). We just need to zero out
217  * that RAM region ourselves before main() runs.
218  *
219  * r2 = current address being zeroed (starts at _sbss)
220  *
221  * r4 = end address (_ebss)
222  *
223  * r3 = the value 0 (what we're writing)
224  *
225  * Same pattern as above: jump to condition check first.
226  */
227
228  ldr r2, =_sbss    /* Load start address of .bss in RAM */
229
230  ldr r4, =_ebss    /* Load end address of .bss in RAM */
231
232  movs r3, #0       /* r3 = 0 (the value to write everywhere) */
233
234  b LoopFillZerobss /* Jump to condition check first */

```

```
226
227
FillZeroBss:
228
    /* Loop body: write one 32-bit zero per iteration */
229
    str r3, [r2]      /* Store 0 into current address (r2 points to RAM) */
230
    adds r2, r2, #4    /* Advance pointer by 4 bytes */
231
232
LoopFillZeroBss:
233
    /* Loop condition: have we zeroed everything up to _ebss? */
234
    cmp r2, r4        /* Compare current pointer against end of .bss */
235
    bcc FillZeroBss    /* Loop back if r2 < r4 (not done yet) */
236
237
238
    /* --- Step 4: Initialize the system clock ---
239
    *
240
    * SystemInit() is a function provided by ST's CMSIS/HAL library (in
241
    * system_stm32f4xx.c). It configures the PLL (Phase-Locked Loop) and
242
    * clock prescalers to run the chip at its target frequency (up to 168 MHz
243
    * on the F407). Without this call, the chip runs from the slow internal
244
    * RC oscillator at 16 MHz with no PLL.
245
    *
246
    * "bl" = Branch with Link. It calls the function AND saves the return
247
    * address in the Link Register (LR / r14), so the function can return
248
```

```

    * here with "bx lr".
249
    */
250
    bl SystemInit
251
252
253
    /* --- Step 5: Run C++ static constructors ---
254
    *
255
    * __libc_init_array() is part of the C runtime library (newlib).
256
    * It iterates over the .init_array section in Flash and calls every
257
    * function pointer stored there. The compiler places constructors for
258
    * global C++ objects there, as well as any functions marked with
259
    * __attribute__((constructor)) in C.
260
    *
261
    * If you're writing pure C with no global objects, this effectively does
262
    * nothing – but it's safe and correct to always call it.
263
    */
264
    bl __libc_init_array
265
266
267
    /* --- Step 6: Call main() ---
268
    *
269
    * Everything is now set up. Jump to the user's application entry point.
270
    * "bl main" is technically a function call (saves return address in LR),

```

```

271     * which is correct – main() could theoretically return, though on a
272
273     * bare-metal embedded system it normally never does.
274
275     */
276
277     bl main
278
279     /* --- Step 7: Trap if main() ever returns ---
280
281     *
282     * On an embedded system, main() returning is almost always a bug –
283
284     * there's nowhere sensible to go back to. This infinite loop prevents
285
286     * the program counter from wandering into random memory and executing
287
288     * garbage instructions, which could cause unpredictable hardware behavior.
289
290     *
291
292     * A debugger can detect the chip is stuck here by examining the PC register.
293
294     */
295
296 LoopForever:
297
298     b LoopForever      /* "b" = unconditional Branch – jumps back to itself forever */
299
300
301
302
303     /* Tell the assembler the size of Reset_Handler (used by debuggers for
304
305     stack unwinding and function boundary detection).
306
307     "." means "current address", so this computes: current_addr - Reset_Handler_start */
308
309

```

```

.size Reset_Handler, .-Reset_Handler
294
295
296
/* =====
297
* DEFAULT INTERRUPT HANDLER
298
* =====
299
*
300
* If an interrupt fires but no specific handler function has been written
301
* for it, the weak alias mechanism (defined at the bottom of this file)
302
* redirects it here. This handler just spins forever in an infinite loop.
303
*
304
* In practice, if you hit this handler unexpectedly, it means:
305
* - An interrupt was enabled that you forgot to write a handler for
306
* - An interrupt fired due to a hardware misconfiguration or a bug
307
*
308
* When debugging, you can set a breakpoint here to catch unexpected interrupts.
309
* Then examine the stack and LR register to figure out which interrupt fired.
310
*
311
* "ax" in the .section directive means:
312
* a = allocatable (takes up space in the binary)
313
* x = executable (contains instructions, not just data)
314
* %progbits means it contains actual program data (not zero-fill like .bss)
315
*/

```

```
316     .section .text.Default_Handler,"ax",%progbits
317
Default_Handler:
318
Infinite_Loop:
319     b Infinite_Loop    /* Spin here forever. The system is in an unknown state.
320
                        A watchdog timer (if configured) will eventually reset
321
                        the chip and give it another chance to run correctly. */
322
    .size Default_Handler, .-Default_Handler
323
324
325
/* =====
326
    * INTERRUPT VECTOR TABLE
327
    * =====
328
    *
329
    * WHAT IS THE VECTOR TABLE?
330
    * On ARM Cortex-M, when an interrupt or exception occurs, the hardware
331
    * automatically looks up a table of function pointers stored at the very
332
    * beginning of Flash memory (address 0x00000000). Each slot in the table
333
    * corresponds to a specific exception or interrupt source. The hardware reads
334
    * the address stored in that slot and jumps to it – this is the "handler".
335
    *
336
    * The table MUST be placed at address 0x00000000 (or at a location pointed
337
    * to by the VTOR register – Vector Table Offset Register – if you move it).
338
```

```

339 * The linker script uses the ".isr_vector" section name to ensure this.
340 *
341 * TABLE LAYOUT:
342 * Slot 0: Initial Stack Pointer value (not a function – it's loaded into SP at reset)
343 * Slot 1: Reset Handler address (where execution begins)
344 * Slot 2+: Exception and interrupt handler addresses
345 *
346 * Each entry is a 32-bit address (.word = 4 bytes).
347 * Entries that say "0" are "reserved" by ST – those interrupt numbers don't
348 * exist on this chip, so the slot is left empty (null pointer).
349 *
350 * "a" in .section means allocatable (included in the binary output).
351 * %progbits means it contains real data.
352 */
353 .section .isr_vector,"a",%progbits
354 .type g_pfnVectors, %object
355 .size g_pfnVectors, .-g_pfnVectors
356
357 g_pfnVectors:
358
359 /* ---- ARM Cortex-M Core Exceptions (positions 0-15) ---- */
360
361 /* These are defined by the ARM architecture itself, not ST.
362
363 Every Cortex-M chip has these same first 16 entries. */

```



```
361
362
363     .word _estack           /* [0] Initial stack pointer value.
                                NOT a handler address – the hardware loads
364                                this directly into SP (r13) on reset.
365                                _estack is the top of RAM. */
366
367
368     .word Reset_Handler    /* [1] Reset exception – runs on power-on/reset.
                                This is the entry point defined above. */
369
370
371     .word NMI_Handler       /* [2] Non-Maskable Interrupt – cannot be
                                disabled by software. Used for critical
372                                faults like clock failure. */
373
374
375     .word HardFault_Handler /* [3] Hard Fault – fires when a fault occurs
                                that has no more specific handler, or when
376                                a fault occurs inside another fault handler.
377                                Common causes: null pointer dereference,
378                                stack overflow, bad instruction. */
379
380
381     .word MemManage_Handler /* [4] Memory Management Fault – MPU (Memory
                                Protection Unit) access violation. Fires
382                                when code tries to access memory it's not
383
```

```

384         allowed to (e.g., executing from RAM when
385
386         the MPU forbids it). */
387
388         .word BusFault_Handler      /* [5] Bus Fault – error on the AHB/APB bus
389
390         during a memory access. Can be caused by
391
392         accessing a non-existent peripheral address
393
394         or bad DMA configuration. */
395
396         .word UsageFault_Handler    /* [6] Usage Fault – undefined instruction,
397
398         illegal unaligned access, invalid state
399
400         transition (e.g., trying to switch to
401
402         ARM mode from Thumb-only Cortex-M). */
403
404         .word 0                     /* [7] Reserved by ARM spec */
405
406         .word 0                     /* [8] Reserved by ARM spec */
407
408         .word 0                     /* [9] Reserved by ARM spec */
409
410         .word 0                     /* [10] Reserved by ARM spec */
411
412         .word SVC_Handler           /* [11] SVCall (Supervisor Call) – triggered by
413
414         the "svc" instruction. Used by RTOS kernels
415
416         (like FreeRTOS) to request privileged
417
418         operations from unprivileged task code. */
419

```

```
406     .word DebugMon_Handler      /* [12] Debug Monitor – fires when a software
407
408                                breakpoint or watchpoint is hit while the
409
410                                debug monitor is active (as opposed to
411
412                                being halted by a hardware debugger). */
413
414     .word 0                    /* [13] Reserved by ARM spec */
415
416
417
418     .word PendSV_Handler       /* [14] PendSV (Pendable Service Call) – a
419
420                                software-triggerable, low-priority interrupt.
421
422                                FreeRTOS and other RTOSes use this for
423
424                                context switching between tasks, because it
425
426                                can be pended (deferred) and will run after
427
428                                all higher-priority interrupts complete. */
429
430
431
432     .word SysTick_Handler      /* [15] SysTick – a simple 24-bit countdown
433
434                                timer built into the Cortex-M core.
435
436                                HAL uses this for the HAL_Delay() 1ms tick.
437
438                                FreeRTOS uses it for the scheduler tick. */
439
440
441
442     /* ---- STM32F407-Specific Peripheral Interrupts (positions 16+) ----
443
444     *
```

```

429 * Everything from here on is specific to the STM32F407 chip design by ST.
430
431 * These are IRQs (Interrupt Requests) from the on-chip peripherals.
432
433 * The number in the comment is the IRQ number (position - 16).
434
435 * In C code you reference these with enum values like USART1_IRQn.
436
437 *
438
439 * In your application code, you define a handler by simply writing a
440
441 * function with the exact same name, e.g.:
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

[illegible]

Used to detect button presses, encoder

pulses, or any digital signal edge. */

```
.word EXTI1_IRQHandler /* IRQ7: EXTI Line 1 – GPIO pin 1 */
```

```
.word EXTI2_IRQHandler /* IRQ8: EXTI Line 2 – GPIO pin 2 */
```

```
.word EXTI3_IRQHandler /* IRQ9: EXTI Line 3 – GPIO pin 3 */
```

```
.word EXTI4_IRQHandler /* IRQ10: EXTI Line 4 – GPIO pin 4 */
```

```
.word DMA1_Stream0_IRQHandler /* IRQ11: DMA1 Stream 0.
```

DMA = Direct Memory Access. The DMA

controller moves data between peripherals

and RAM without involving the CPU, freeing

the CPU to do other work. Each "stream"

is an independent DMA channel.

This fires when the transfer completes,

or when a transfer error occurs. */

```
.word DMA1_Stream1_IRQHandler /* IRQ12: DMA1 Stream 1 */
```

```
.word DMA1_Stream2_IRQHandler /* IRQ13: DMA1 Stream 2 */
```

```
.word DMA1_Stream3_IRQHandler /* IRQ14: DMA1 Stream 3 */
```

```
.word DMA1_Stream4_IRQHandler /* IRQ15: DMA1 Stream 4 */
```

```
.word DMA1_Stream5_IRQHandler /* IRQ16: DMA1 Stream 5 */
```

```
.word DMA1_Stream6_IRQHandler /* IRQ17: DMA1 Stream 6 */
```

```
496
497
498     .word ADC_IRQHandler          /* IRQ18: ADC (Analog-to-Digital Converter)
499
500                                     global interrupt. The F407 has 3 ADCs.
501
502                                     Fires on conversion complete, overrun,
503
504                                     or injected channel end. */
505
506
507
508     .word CAN1_TX_IRQHandler      /* IRQ19: CAN1 Transmit – CAN (Controller Area
509
510                                     Network) is a robust serial protocol used
511
512                                     in automotive and industrial systems.
513
514                                     This fires when a CAN frame finishes
515
516                                     transmitting (mailbox empty). */
517
518
519     .word CAN1_RX0_IRQHandler     /* IRQ20: CAN1 Receive FIFO 0 – fires when a
520
521                                     CAN frame is received into FIFO 0. */
522
523
524
525     .word CAN1_RX1_IRQHandler     /* IRQ21: CAN1 Receive FIFO 1 */
526
527
528
529     .word CAN1_SCE_IRQHandler     /* IRQ22: CAN1 Status Change / Error –
530
531                                     fires on bus-off, error passive, or
532
533                                     last error code changes. */
534
535
536
537     .word EXTI9_5_IRQHandler      /* IRQ23: EXTI Lines 5–9 – GPIO pins 5,6,7,8,9
538
```

share a single interrupt. Your handler must

check the EXTI_PR register to determine

which specific pin triggered it. */

```
.word TIM1_BRK_TIM9_IRQHandler /* IRQ24: TIM1 Break + TIM9 global.
```

TIM1 is an "advanced" timer with PWM

features. The Break input shuts down PWM

outputs on a fault condition (e.g., overcurrent).

TIM9 is a general-purpose timer. */

```
.word TIM1_UP_TIM10_IRQHandler /* IRQ25: TIM1 Update (counter overflow/underflow)
```

+ TIM10 global interrupt. */

```
.word TIM1_TRG_COM_TIM11_IRQHandler /* IRQ26: TIM1 Trigger & Commutation
```

(used in 3-phase motor control) + TIM11 global. */

```
.word TIM1_CC_IRQHandler /* IRQ27: TIM1 Capture/Compare – fires when
```

a timer channel captures an input signal

timestamp or matches an output compare value. */

```
.word TIM2_IRQHandler /* IRQ28: TIM2 global – 32-bit general purpose
```

timer, useful for precise time measurement. */


```
541      .word TIM3_IRQHandler          /* IRQ29: TIM3 global – 16-bit general purpose timer */
542
543      .word TIM4_IRQHandler          /* IRQ30: TIM4 global – 16-bit general purpose timer */
544
545      .word I2C1_EV_IRQHandler       /* IRQ31: I2C1 Event – I2C is a 2-wire serial
546                                     bus (SDA + SCL) used for sensors, EEPROMs,
547                                     displays, etc. The Event interrupt fires on
548                                     address match, byte received/sent, STOP detected. */
549
550      .word I2C1_ER_IRQHandler       /* IRQ32: I2C1 Error – bus error, arbitration
551                                     loss, NACK received, etc. */
552
553      .word I2C2_EV_IRQHandler       /* IRQ33: I2C2 Event */
554
555      .word I2C2_ER_IRQHandler       /* IRQ34: I2C2 Error */
556
557      .word SPI1_IRQHandler          /* IRQ35: SPI1 global – SPI (Serial Peripheral
558                                     Interface) is a fast 4-wire bus (MOSI, MISO,
559                                     SCK, CS) used for SD cards, displays,
560                                     ADCs, sensors, etc. */
561
562      .word SPI2_IRQHandler          /* IRQ36: SPI2 global */
563
564      .word USART1_IRQHandler        /* IRQ37: USART1 – Universal Synchronous/
```

Asynchronous Receiver-Transmitter. This is

your standard serial/UART. Fires on byte

received, transmit buffer empty, etc.

Commonly used for debug output or talking

to a PC over USB-serial. */

```
.word USART2_IRQHandler    /* IRQ38: USART2 global */
```

```
.word USART3_IRQHandler    /* IRQ39: USART3 global */
```

```
.word EXTI15_10_IRQHandler  /* IRQ40: EXTI Lines 10-15 – GPIO pins 10-15.
```

```
    Same shared-interrupt situation as EXTI9_5. */
```

```
.word RTC_Alarm_IRQHandler  /* IRQ41: RTC Alarm A or B through EXTI –
```

```
    Wake up / trigger at a specific date/time. */
```

```
.word OTG_FS_WKUP_IRQHandler /* IRQ42: USB OTG Full-Speed Wakeup – wakes
```

```
    the chip from low-power mode when USB
```

```
    activity is detected. */
```

```
.word TIM8_BRK_TIM12_IRQHandler /* IRQ43: TIM8 Break + TIM12 global */
```

```
.word TIM8_UP_TIM13_IRQHandler /* IRQ44: TIM8 Update + TIM13 global */
```

```
.word TIM8_TRG_COM_TIM14_IRQHandler /* IRQ45: TIM8 Trigger/Commutation + TIM14 */
```

```
.word TIM8_CC_IRQHandler     /* IRQ46: TIM8 Capture/Compare */
```

```
586     .word DMA1_Stream7_IRQHandler /* IRQ47: DMA1 Stream 7 */
587
588     .word FSMC_IRQHandler          /* IRQ48: FSMC (Flexible Static Memory
589                                     Controller) – used to interface with
590                                     external parallel memories like SRAM,
591                                     NOR Flash, and LCD controllers. */
592
593     .word SDIO_IRQHandler          /* IRQ49: SDIO – SD card interface interrupt.
594                                     Fires on transfer complete, FIFO errors, etc. */
595
596     .word TIM5_IRQHandler          /* IRQ50: TIM5 global – 32-bit general purpose timer */
597
598     .word SPI3_IRQHandler          /* IRQ51: SPI3 global */
599
600     .word UART4_IRQHandler         /* IRQ52: UART4 global – Note: UART (not USART).
601                                     UARTs lack the synchronous clock line that
602                                     USARTs have. Still async serial. */
603
604     .word UART5_IRQHandler         /* IRQ53: UART5 global */
605
606     .word TIM6_DAC_IRQHandler      /* IRQ54: TIM6 global + DAC underrun error.
607                                     TIM6 is often used as the trigger source
608                                     for the DAC (Digital-to-Analog Converter).
```

Underrun = DAC ran out of data to output. */

609

610

.word TIM7_IRQHandler /* IRQ55: TIM7 global – basic timer, often used

611

as a timebase for DAC or periodic callbacks */

612

613

.word DMA2_Stream0_IRQHandler /* IRQ56: DMA2 Stream 0 */

614

.word DMA2_Stream1_IRQHandler /* IRQ57: DMA2 Stream 1 */

615

.word DMA2_Stream2_IRQHandler /* IRQ58: DMA2 Stream 2 */

616

.word DMA2_Stream3_IRQHandler /* IRQ59: DMA2 Stream 3 */

617

.word DMA2_Stream4_IRQHandler /* IRQ60: DMA2 Stream 4 */

618

619

.word ETH_IRQHandler /* IRQ61: Ethernet MAC global interrupt –

620

the STM32F407 has a built-in 10/100 Mbps

621

Ethernet MAC (Media Access Controller).

622

Fires on frame received/sent, etc. */

623

624

.word ETH_WKUP_IRQHandler /* IRQ62: Ethernet Wakeup through EXTI –

625

wake from low-power mode on network activity */

626

627

.word CAN2_TX_IRQHandler /* IRQ63: CAN2 Transmit */

628

.word CAN2_RX0_IRQHandler /* IRQ64: CAN2 Receive FIFO 0 */

629

.word CAN2_RX1_IRQHandler /* IRQ65: CAN2 Receive FIFO 1 */

630

.word CAN2_SCE_IRQHandler /* IRQ66: CAN2 Status/Error */

631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653

```
.word OTG_FS_IRQHandler      /* IRQ67: USB OTG Full-Speed global interrupt –  
  
                                the STM32F407 has a built-in USB 2.0  
  
                                Full-Speed (12 Mbps) OTG controller.  
  
                                OTG = On-The-Go (can be host or device). */  
  
.word DMA2_Stream5_IRQHandler /* IRQ68: DMA2 Stream 5 */  
.word DMA2_Stream6_IRQHandler /* IRQ69: DMA2 Stream 6 */  
.word DMA2_Stream7_IRQHandler /* IRQ70: DMA2 Stream 7 */  
  
.word USART6_IRQHandler      /* IRQ71: USART6 global */  
  
.word I2C3_EV_IRQHandler      /* IRQ72: I2C3 Event */  
.word I2C3_ER_IRQHandler      /* IRQ73: I2C3 Error */  
  
.word OTG_HS_EP1_OUT_IRQHandler /* IRQ74: USB OTG High-Speed Endpoint 1 OUT –  
  
                                the F407 also has a USB HS (480 Mbps)  
  
                                controller (though it typically needs an  
  
                                external HS PHY chip). This handles data  
  
                                flowing OUT from host to device on EP1. */  
  
.word OTG_HS_EP1_IN_IRQHandler /* IRQ75: USB OTG HS Endpoint 1 IN –
```

data flowing IN from device to host on EP1. */

654

655

.word OTG_HS_WKUP_IRQHandler /* IRQ76: USB OTG HS Wakeup through EXTI */

656

.word OTG_HS_IRQHandler /* IRQ77: USB OTG HS global interrupt */

657

658

.word DCMI_IRQHandler /* IRQ78: DCMI (Digital Camera Interface) –

659

parallel camera interface for connecting

660

OV7670-style camera modules directly. */

661

662

.word CRYPT_IRQHandler /* IRQ79: Cryptographic processor global –

663

hardware AES, DES, Triple-DES acceleration. */

664

665

.word HASH_RNG_IRQHandler /* IRQ80: Hash (SHA-1, MD5) + RNG (Random Number

666

Generator) global interrupt. The hardware

667

RNG uses analog noise to generate true

668

random numbers – useful for cryptography. */

669

670

.word FPU_IRQHandler /* IRQ81: FPU (Floating-Point Unit) global –

671

fires on FPU exceptions like divide-by-zero,

672

overflow, underflow, or invalid operation,

673

if those exception traps are enabled. */

674

675

/* Slots 82-87: Reserved by ST (these IRQ numbers don't exist on F407) */

```
676     .word 0
677
678     .word 0
679
680     .word 0
681
682     .word 0
683
684     .word LCD_TFT_IRQHandler      /* IRQ88: LTDC (LCD-TFT Display Controller) –
685                                     the F407 has a hardware layer compositor
686                                     for driving RGB TFT displays. This fires
687                                     on frame sync, line events, etc. */
688
689     .word LCD_TFT_1_IRQHandler    /* IRQ89: LTDC Error interrupt – fires on FIFO
690                                     underrun or transfer error in the display
691                                     controller. */
692
693
694     /* =====
695     * WEAK ALIASES FOR ALL INTERRUPT HANDLERS
696     * =====
697
698     *
699     * WHAT IS A WEAK ALIAS?
```

```
* The directives below do two things for each handler:
699
*
700
* .weak SomeHandler
701
* Makes SomeHandler a "weak" symbol. If your application code defines
702
* a function with the exact same name, the linker uses YOUR version and
703
* ignores this one. If you DON'T define it, the linker keeps this one.
704
*
705
* .thumb_set SomeHandler, Default_Handler
706
* Makes SomeHandler an alias (another name) for Default_Handler.
707
* Combined with .weak, this means: "by default, SomeHandler points to
708
* Default_Handler (the infinite loop), but any user-defined function
709
* called SomeHandler will override this."
710
*
711
* THE PRACTICAL RESULT:
712
* You only need to write handlers for interrupts you actually use.
713
* All others silently redirect to the infinite loop, making debugging easier
714
* (you can set a breakpoint in Default_Handler to catch any unhandled IRQ).
715
*
716
* EXAMPLE:
717
* If you want to handle UART1 bytes:
718
*
719
* void USART1_IRQHandler(void) { // exact name match required!
720
*     char c = USART1->DR; // read the received byte
```



```
721
*      my_buffer_push(c);
722
*   }
723
*
724
* The linker sees your strong definition and uses it instead of the
725
* weak alias to Default_Handler below.
726
*/
727
728
/* Core ARM exception handlers */
729
.weak      NMI_Handler
730
.thumb_set NMI_Handler,Default_Handler
731
732
.weak      HardFault_Handler
733
.thumb_set HardFault_Handler,Default_Handler
734
735
.weak      MemManage_Handler
736
.thumb_set MemManage_Handler,Default_Handler
737
738
.weak      BusFault_Handler
739
.thumb_set BusFault_Handler,Default_Handler
740
741
.weak      UsageFault_Handler
742
.thumb_set UsageFault_Handler,Default_Handler
743
```

```
744
    .weak      SVC_Handler

745
    .thumb_set SVC_Handler,Default_Handler

746

747
    .weak      DebugMon_Handler

748
    .thumb_set DebugMon_Handler,Default_Handler

749

750
    .weak      PendSV_Handler

751
    .thumb_set PendSV_Handler,Default_Handler

752

753
    .weak      SysTick_Handler

754
    .thumb_set SysTick_Handler,Default_Handler

755

756
    /* STM32F407 peripheral interrupt handlers */

757
    .weak      WWDG_IRQHandler

758
    .thumb_set WWDG_IRQHandler,Default_Handler

759

760
    .weak      PVD_IRQHandler

761
    .thumb_set PVD_IRQHandler,Default_Handler

762

763
    .weak      TAMP_STAMP_IRQHandler

764
    .thumb_set TAMP_STAMP_IRQHandler,Default_Handler

765
```

```
766      .weak      RTC_WKUP_IRQHandler
767      .thumb_set  RTC_WKUP_IRQHandler,Default_Handler
768
769      .weak      RCC_IRQHandler
770      .thumb_set  RCC_IRQHandler,Default_Handler
771
772      .weak      EXTI0_IRQHandler
773      .thumb_set  EXTI0_IRQHandler,Default_Handler
774
775      .weak      EXTI1_IRQHandler
776      .thumb_set  EXTI1_IRQHandler,Default_Handler
777
778      .weak      EXTI2_IRQHandler
779      .thumb_set  EXTI2_IRQHandler,Default_Handler
780
781      .weak      EXTI3_IRQHandler
782      .thumb_set  EXTI3_IRQHandler,Default_Handler
783
784      .weak      EXTI4_IRQHandler
785      .thumb_set  EXTI4_IRQHandler,Default_Handler
786
787      .weak      DMA1_Stream0_IRQHandler
788
```

```
.thumb_set DMA1_Stream0_IRQHandler,Default_Handler
```

789

790

```
.weak      DMA1_Stream1_IRQHandler
```

791

```
.thumb_set DMA1_Stream1_IRQHandler,Default_Handler
```

792

793

```
.weak      DMA1_Stream2_IRQHandler
```

794

```
.thumb_set DMA1_Stream2_IRQHandler,Default_Handler
```

795

796

```
.weak      DMA1_Stream3_IRQHandler
```

797

```
.thumb_set DMA1_Stream3_IRQHandler,Default_Handler
```

798

799

```
.weak      DMA1_Stream4_IRQHandler
```

800

```
.thumb_set DMA1_Stream4_IRQHandler,Default_Handler
```

801

802

```
.weak      DMA1_Stream5_IRQHandler
```

803

```
.thumb_set DMA1_Stream5_IRQHandler,Default_Handler
```

804

805

```
.weak      DMA1_Stream6_IRQHandler
```

806

```
.thumb_set DMA1_Stream6_IRQHandler,Default_Handler
```

807

808

```
.weak      ADC_IRQHandler
```

809

```
.thumb_set ADC_IRQHandler,Default_Handler
```

810

```
811      .weak      CAN1_TX_IRQHandler
812      .thumb_set CAN1_TX_IRQHandler,Default_Handler
813
814      .weak      CAN1_RX0_IRQHandler
815      .thumb_set CAN1_RX0_IRQHandler,Default_Handler
816
817      .weak      CAN1_RX1_IRQHandler
818      .thumb_set CAN1_RX1_IRQHandler,Default_Handler
819
820      .weak      CAN1_SCE_IRQHandler
821      .thumb_set CAN1_SCE_IRQHandler,Default_Handler
822
823      .weak      EXTI9_5_IRQHandler
824      .thumb_set EXTI9_5_IRQHandler,Default_Handler
825
826      .weak      TIM1_BRK_TIM9_IRQHandler
827      .thumb_set TIM1_BRK_TIM9_IRQHandler,Default_Handler
828
829      .weak      TIM1_UP_TIM10_IRQHandler
830      .thumb_set TIM1_UP_TIM10_IRQHandler,Default_Handler
831
832      .weak      TIM1_TRG_COM_TIM11_IRQHandler
833
```

```
.thumb_set TIM1_TRG_COM_TIM11_IRQHandler,Default_Handler
```

834

835

```
.weak      TIM1_CC_IRQHandler
```

836

```
.thumb_set TIM1_CC_IRQHandler,Default_Handler
```

837

838

```
.weak      TIM2_IRQHandler
```

839

```
.thumb_set TIM2_IRQHandler,Default_Handler
```

840

841

```
.weak      TIM3_IRQHandler
```

842

```
.thumb_set TIM3_IRQHandler,Default_Handler
```

843

844

```
.weak      TIM4_IRQHandler
```

845

```
.thumb_set TIM4_IRQHandler,Default_Handler
```

846

847

```
.weak      I2C1_EV_IRQHandler
```

848

```
.thumb_set I2C1_EV_IRQHandler,Default_Handler
```

849

850

```
.weak      I2C1_ER_IRQHandler
```

851

```
.thumb_set I2C1_ER_IRQHandler,Default_Handler
```

852

853

```
.weak      I2C2_EV_IRQHandler
```

854

```
.thumb_set I2C2_EV_IRQHandler,Default_Handler
```

855

```
856      .weak      I2C2_ER_IRQHandler
857      .thumb_set I2C2_ER_IRQHandler,Default_Handler
858
859      .weak      SPI1_IRQHandler
860      .thumb_set SPI1_IRQHandler,Default_Handler
861
862      .weak      SPI2_IRQHandler
863      .thumb_set SPI2_IRQHandler,Default_Handler
864
865      .weak      USART1_IRQHandler
866      .thumb_set USART1_IRQHandler,Default_Handler
867
868      .weak      USART2_IRQHandler
869      .thumb_set USART2_IRQHandler,Default_Handler
870
871      .weak      USART3_IRQHandler
872      .thumb_set USART3_IRQHandler,Default_Handler
873
874      .weak      EXTI15_10_IRQHandler
875      .thumb_set EXTI15_10_IRQHandler,Default_Handler
876
877      .weak      RTC_Alarm_IRQHandler
878
```

```
.thumb_set RTC_Alarm_IRQHandler,Default_Handler
```

879

880

```
.weak      OTG_FS_WKUP_IRQHandler
```

881

```
.thumb_set OTG_FS_WKUP_IRQHandler,Default_Handler
```

882

883

```
.weak      TIM8_BRK_TIM12_IRQHandler
```

884

```
.thumb_set TIM8_BRK_TIM12_IRQHandler,Default_Handler
```

885

886

```
.weak      TIM8_UP_TIM13_IRQHandler
```

887

```
.thumb_set TIM8_UP_TIM13_IRQHandler,Default_Handler
```

888

889

```
.weak      TIM8_TRG_COM_TIM14_IRQHandler
```

890

```
.thumb_set TIM8_TRG_COM_TIM14_IRQHandler,Default_Handler
```

891

892

```
.weak      TIM8_CC_IRQHandler
```

893

```
.thumb_set TIM8_CC_IRQHandler,Default_Handler
```

894

895

```
.weak      DMA1_Stream7_IRQHandler
```

896

```
.thumb_set DMA1_Stream7_IRQHandler,Default_Handler
```

897

898

```
.weak      FSMC_IRQHandler
```

899

```
.thumb_set FSMC_IRQHandler,Default_Handler
```

900


```
901
    .weak      SDIO_IRQHandler
902
    .thumb_set SDIO_IRQHandler,Default_Handler
903
904
    .weak      TIM5_IRQHandler
905
    .thumb_set TIM5_IRQHandler,Default_Handler
906
907
    .weak      SPI3_IRQHandler
908
    .thumb_set SPI3_IRQHandler,Default_Handler
909
910
    .weak      UART4_IRQHandler
911
    .thumb_set UART4_IRQHandler,Default_Handler
912
913
    .weak      UART5_IRQHandler
914
    .thumb_set UART5_IRQHandler,Default_Handler
915
916
    .weak      TIM6_DAC_IRQHandler
917
    .thumb_set TIM6_DAC_IRQHandler,Default_Handler
918
919
    .weak      TIM7_IRQHandler
920
    .thumb_set TIM7_IRQHandler,Default_Handler
921
922
    .weak      DMA2_Stream0_IRQHandler
923
```

```
.thumb_set DMA2_Stream0_IRQHandler,Default_Handler
```

924

925

```
.weak      DMA2_Stream1_IRQHandler
```

926

```
.thumb_set DMA2_Stream1_IRQHandler,Default_Handler
```

927

928

```
.weak      DMA2_Stream2_IRQHandler
```

929

```
.thumb_set DMA2_Stream2_IRQHandler,Default_Handler
```

930

931

```
.weak      DMA2_Stream3_IRQHandler
```

932

```
.thumb_set DMA2_Stream3_IRQHandler,Default_Handler
```

933

934

```
.weak      DMA2_Stream4_IRQHandler
```

935

```
.thumb_set DMA2_Stream4_IRQHandler,Default_Handler
```

936

937

```
.weak      ETH_IRQHandler
```

938

```
.thumb_set ETH_IRQHandler,Default_Handler
```

939

940

```
.weak      ETH_WKUP_IRQHandler
```

941

```
.thumb_set ETH_WKUP_IRQHandler,Default_Handler
```

942

943

```
.weak      CAN2_TX_IRQHandler
```

944

```
.thumb_set CAN2_TX_IRQHandler,Default_Handler
```

945

```
946      .weak      CAN2_RX0_IRQHandler
947      .thumb_set CAN2_RX0_IRQHandler,Default_Handler
948
949      .weak      CAN2_RX1_IRQHandler
950      .thumb_set CAN2_RX1_IRQHandler,Default_Handler
951
952      .weak      CAN2_SCE_IRQHandler
953      .thumb_set CAN2_SCE_IRQHandler,Default_Handler
954
955      .weak      OTG_FS_IRQHandler
956      .thumb_set OTG_FS_IRQHandler,Default_Handler
957
958      .weak      DMA2_Stream5_IRQHandler
959      .thumb_set DMA2_Stream5_IRQHandler,Default_Handler
960
961      .weak      DMA2_Stream6_IRQHandler
962      .thumb_set DMA2_Stream6_IRQHandler,Default_Handler
963
964      .weak      DMA2_Stream7_IRQHandler
965      .thumb_set DMA2_Stream7_IRQHandler,Default_Handler
966
967      .weak      USART6_IRQHandler
968
```

```
.thumb_set USART6_IRQHandler,Default_Handler
```

969

970

```
.weak      I2C3_EV_IRQHandler
```

971

```
.thumb_set I2C3_EV_IRQHandler,Default_Handler
```

972

973

```
.weak      I2C3_ER_IRQHandler
```

974

```
.thumb_set I2C3_ER_IRQHandler,Default_Handler
```

975

976

```
.weak      OTG_HS_EP1_OUT_IRQHandler
```

977

```
.thumb_set OTG_HS_EP1_OUT_IRQHandler,Default_Handler
```

978

979

```
.weak      OTG_HS_EP1_IN_IRQHandler
```

980

```
.thumb_set OTG_HS_EP1_IN_IRQHandler,Default_Handler
```

981

982

```
.weak      OTG_HS_WKUP_IRQHandler
```

983

```
.thumb_set OTG_HS_WKUP_IRQHandler,Default_Handler
```

984

985

```
.weak      OTG_HS_IRQHandler
```

986

```
.thumb_set OTG_HS_IRQHandler,Default_Handler
```

987

988

```
.weak      DCMI_IRQHandler
```

989

```
.thumb_set DCMI_IRQHandler,Default_Handler
```

990

```
991     .weak      CRYPT_IRQHandler
992
993     .thumb_set CRYPT_IRQHandler,Default_Handler
994
995     .weak      HASH_RNG_IRQHandler
996
997     .thumb_set HASH_RNG_IRQHandler,Default_Handler
998
999     .thumb_set FPU_IRQHandler,Default_Handler
1000
1001     .weak      LCD_TFT_IRQHandler
1002
1003     .thumb_set LCD_TFT_IRQHandler,Default_Handler
1004
1005     .weak      LCD_TFT_1_IRQHandler
1006
1007     .thumb_set LCD_TFT_1_IRQHandler,Default_Handler
1008
1009     /* SystemInit is also declared weak here. This means if you don't provide
1010     your own SystemInit() (e.g., if you're not using ST's HAL/CMSIS), the
1011     linker won't complain – it just won't call anything for clock setup.
1012     Normally you DO want SystemInit() from system_stm32f4xx.c to run. */
1013     .weak      SystemInit
1014
1015     /***** (C) COPYRIGHT STMicroelectronics *****/
1016     *****END OF FILE*****/
```

